T1-Introducció

SISTEMES OPERATIUS
GRAU EN ENGINYERIA INFORMÀTICA

Autors: Professors de Sistemes Operatius del
Departament d'Arquitectura de Computadors
(UPC- BarcelonaTech)

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Llicència



«By Universitat Politècnica de Catalunya - BarcelonaTech (UPC), Any 2025»

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.ca

Índex

- El rol del SO
- Serveis que ofereix el S.O.
- Maneres d'accedir al kernel (Tema 8 EC)
  - Modes d'execució
  - Interrupcions, excepcions i crides a sistema
- Crides a sistema (Tema 8 EC)
  - Generació d'executables
  - Requeriments de les crides a sistema
  - Llibreries



EL ROL DEL S.O.

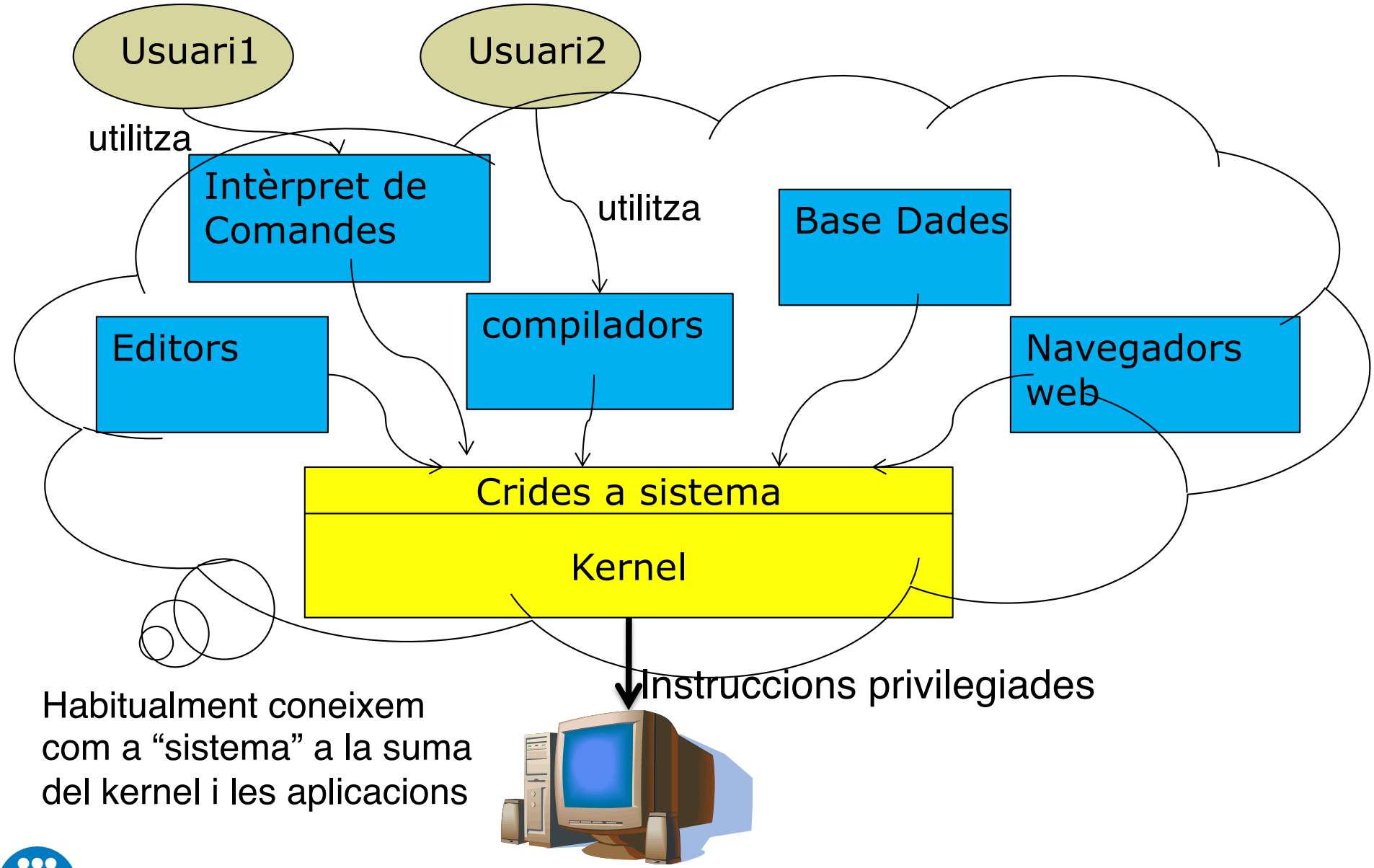


Què és un SO?

- El Sistema Operatiu és un software que controla els recursos disponibles del sistema hardware que volem utilitzar i que fa d'intermediari entre les aplicacions i el hardware
  - **Internament** defineix estructures de dades per a gestionar el HW i algorismes per a decidir com utilizar-lo
  - **Externament** ofereix un conjunt de funcions per a accedir a la seva funcionalitat (o serveis) de gestió de recursos

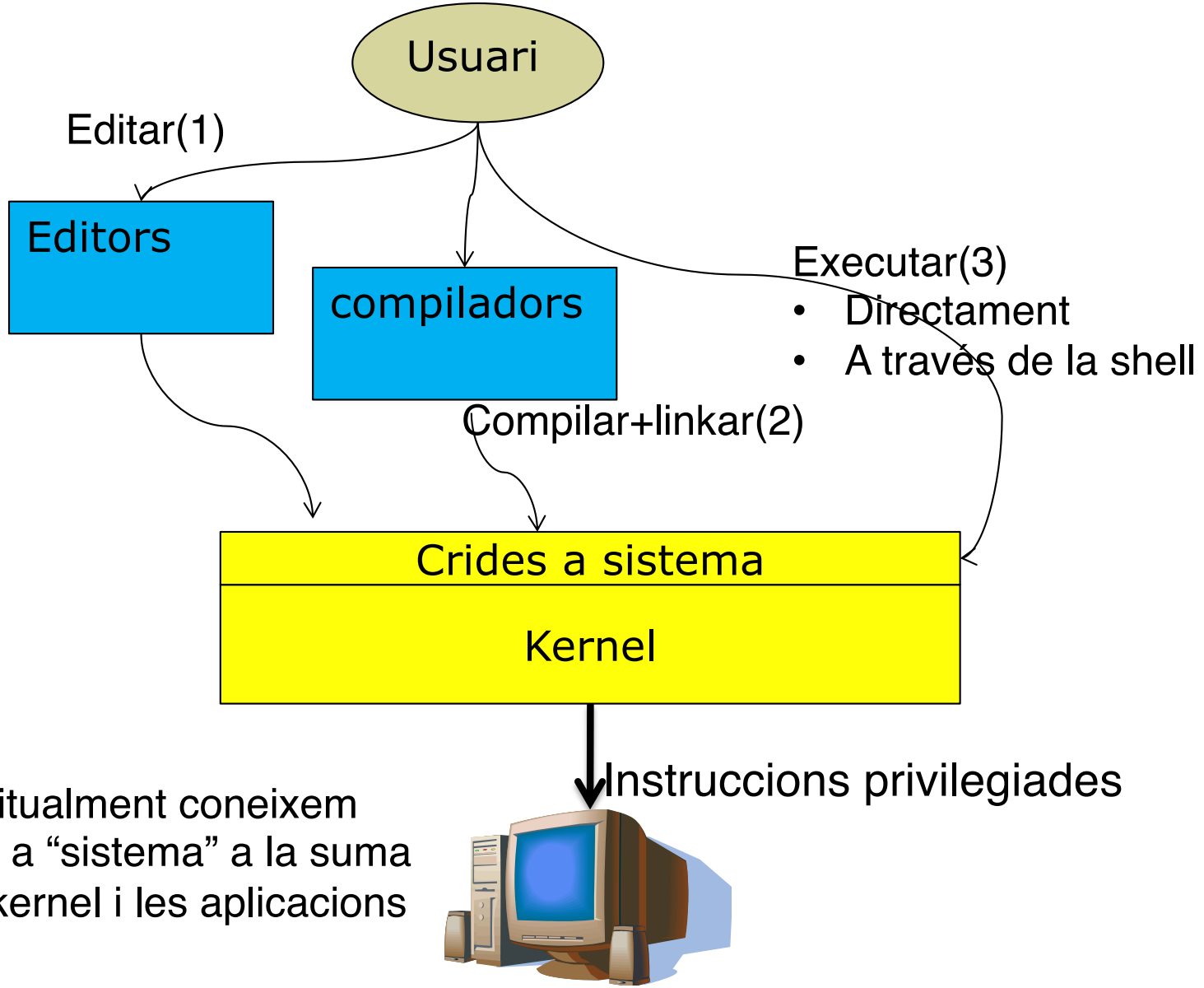


Components del sistema



Habitualment coneixem com a “sistema” a la suma del kernel i les aplicacions

Utilització del sistema: Desenvolupament i execució de programes



Habitualment coneixem com a “sistema” a la suma del kernel i les aplicacions

Utilització del sistema



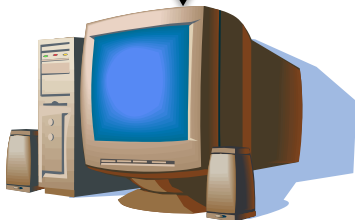
```
# gedit p1.c
# gcc –o p1 p1.c
# p1
```

Normalment utilitzem un **entorn de treball**, l'anomenrarem shell o intèrpret de comandes.

Crides a sistema

Kernel

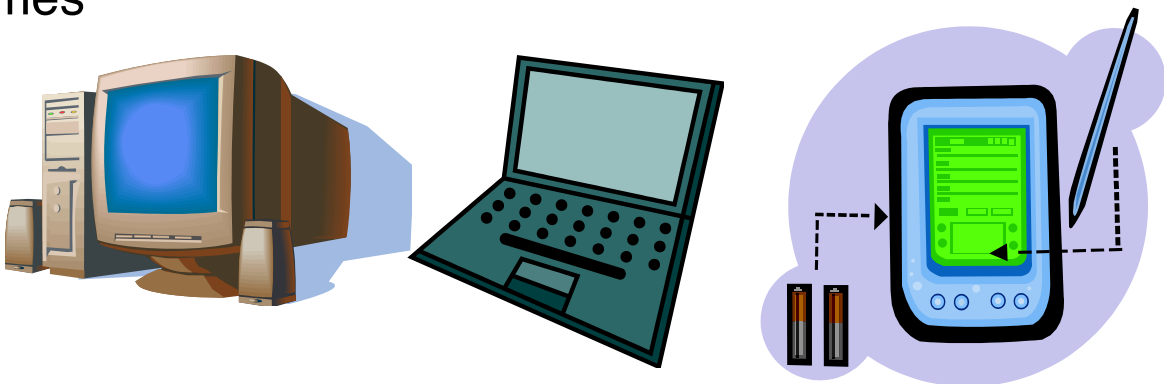
Instruccions privilegiades



Habitualment coneixem com a "sistema" a la suma del kernel i les aplicacions

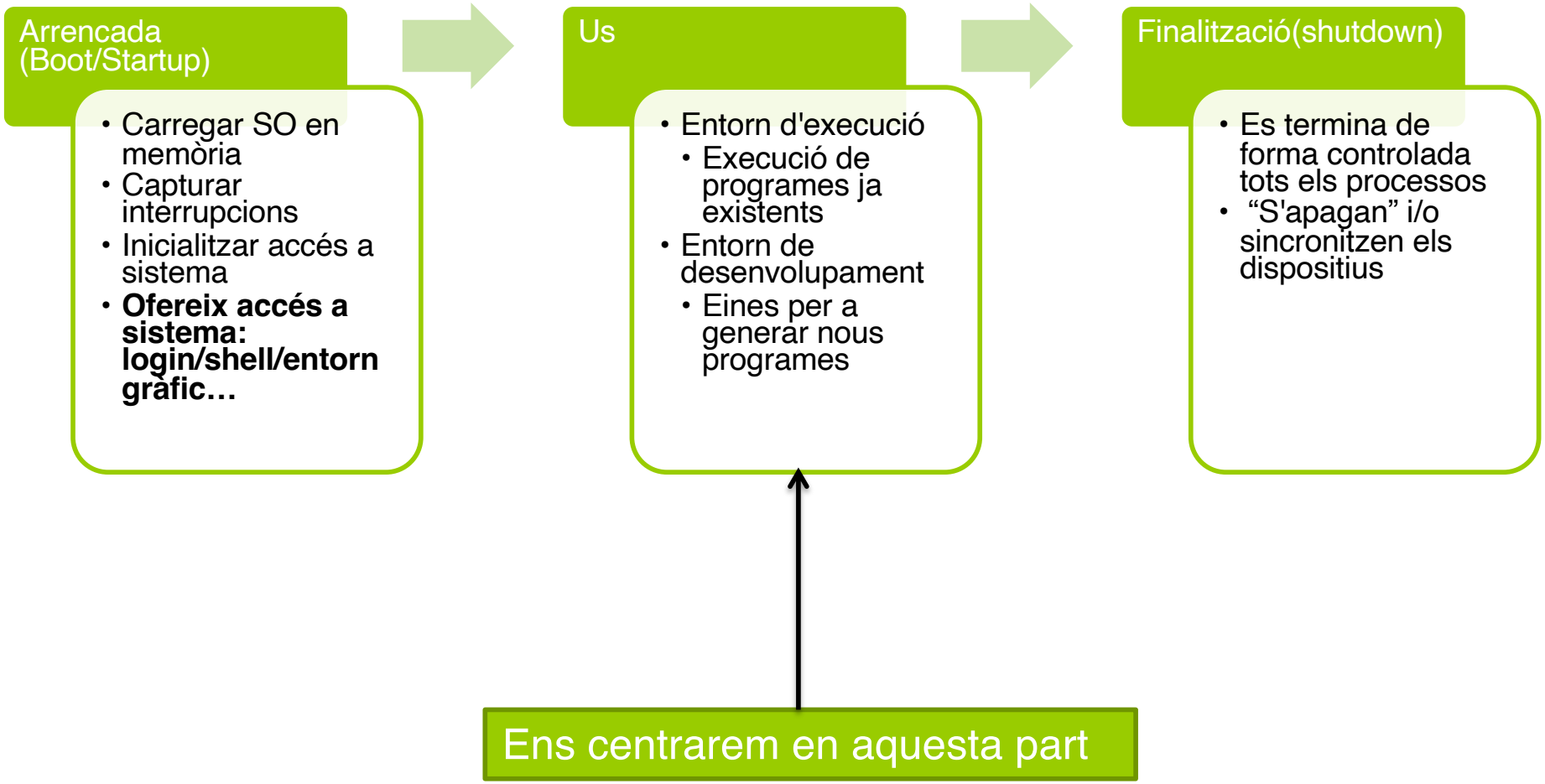
Què esperem del S.O?

- Ofereix un **entorn "usable"**
  - Abstraeix als usuaris de las diferències que hi ha entre els diferents sistemes



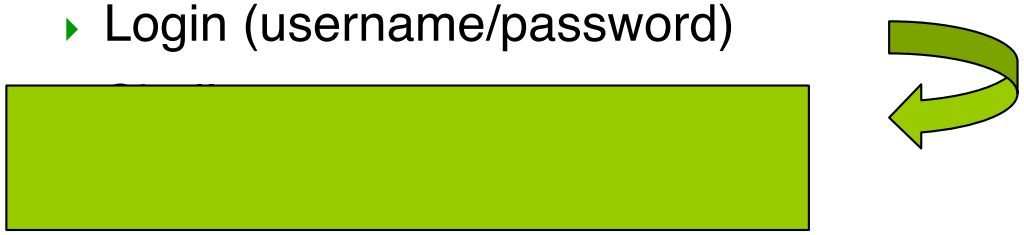
- Ofereix un **entorn segur**
  - Protegeix el HW d'accessos incorrectes i a uns usuaris dels altres
- Ofereix un **entorn eficient**
  - Proporciona un us eficient dels recursos del sistema
  - Múltiples usuaris accedeixen a un mateix sistema i tenen una sensació d'accés en "exclusiva"

Què ofereix el SO?



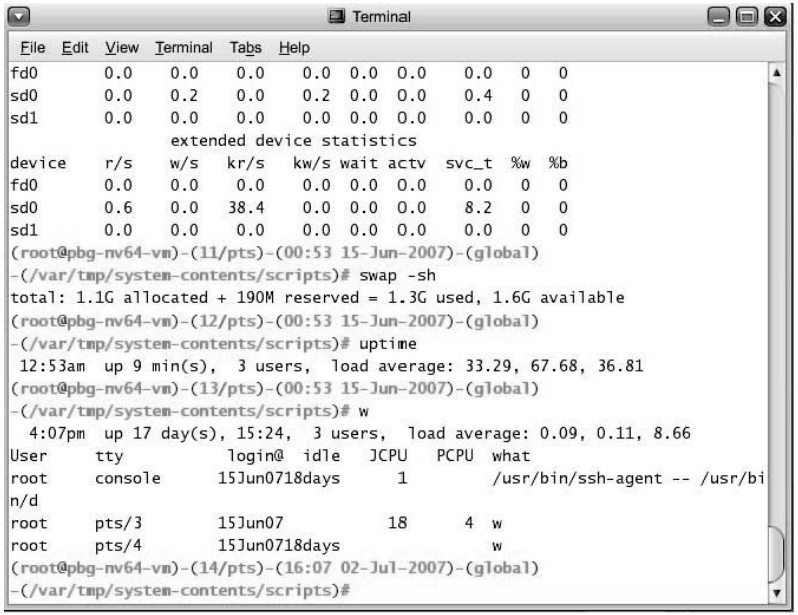
Arrencada del sistema

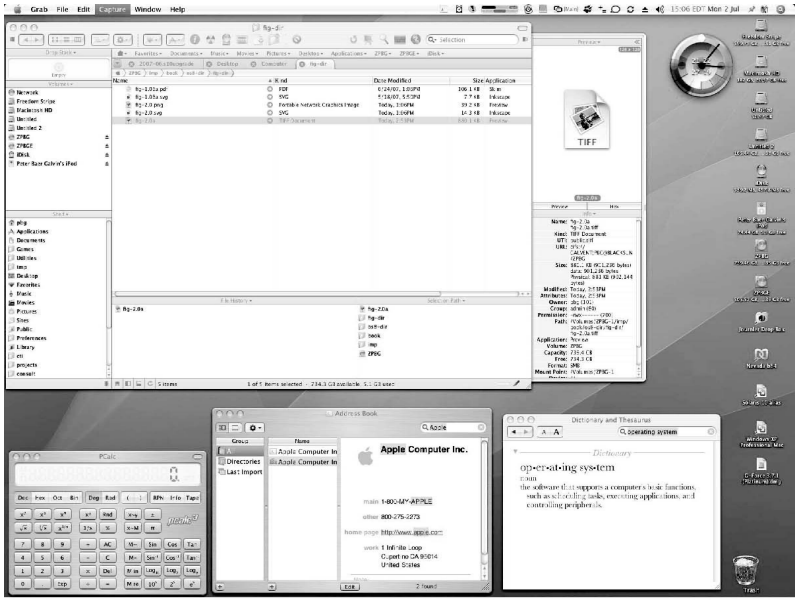
- El hardware carrega el SO (o una part) en arrencar. Es coneix com a *boot*
  - **El sistema pot tenir més d'un SO instal.lat (en el disc) pero només un en execució**
  - El SO es copia en memòria (tot o una part d'ell)
  - Inicialitza les estructures de dades necessàries (hardware/software) per a controlar la màquina i oferir els serveis
  - Les interrupcions hardware són capturades pel SO
  - Al final el sistema posa en marxa un programa que permet accedir al sistema
    - Login (username/password)



Entorn de desenvolupament

- El SO ofereix, com a part dels seus serveis, un entorn de treball **interactiu**. Bàsicament pot ser de dos tipus: intèrprete de comandes o entorn gràfic
- Depenent del sistema, aquest entorn pot formar part del *kernel* o pot ser un programa separat. En qualsevol cas el trobem com a part del sistema per a poder utilizar-lo.





Entorn “usable”, segur i eficient

- Durant el funcionament del sistema, aquest són els seus tres objetius bàsics
  - Usabilitat
  - Seguretat
  - Eficiència
- També hauran de ser-ho els vostres programes
  - Usables:
    - Si l’usuari no l’executa correctament, haurà de donar un missatge indicant-ho que sigui clar i útil
    - Si alguna funció d’usuari o crida a sistema falla, el programa haura de donar un missatge clar i útil
  - Segur:
    - El programa no pot (o no hauria de) “fallar” si l’usuari l’utilitza de forma incorrecta, ha de controlar tote les fonts d’error.
    - I de cap manerar fer fallar el sistema
  - Eficient:
    - Ens hem de plantejar la millor manera d’aprofitar els recursos de la máquina, tenint en compte que els recursos que té l’usuari són limitats (execució en entorn compartit)



(Explicat a EC)

FORMES D'ACCEDIR AL KERNEL



Com oferir un entorn segur: “Modes” d’execució

- El SO necessita una manera de garantitzar la seva seguretat, la del hardware i la de la resta de processos
- Necessitem instruccions de llenguatge màquina privilegiades que només pot executar el kernel
- El HW coneix quan s’está executant el kernel i quan una aplicació d’usuari. Hi ha una instrucció de llenguatge màquina per a passar d’un mode a l’altre.
- **El SO s’executa en un mode d’execució privilegiat.**
  - **Mínim 2 modes (hi poden haver més)**
    - **Mode d’execució NO-privilegiat , *user mode***
    - **Mode d’execució privilegiat, *kernel mode***
  - **Hi han parts de la memòria només accessibles en mode privilegiat i determinades instruccions de llenguatge màquina només es poden executar en mode privilegiat**

- Objetiu: Entendre què són els modes d’execució i per què els necessitem

Quan s'executa codi de kernel?

- Quan una aplicació executa una crida a sistema
- Quan una aplicació provoca una excepció
- Quan un dispositiu provoca una interrupció

- Aquests events podrien no passar, i el SO no s'executaria → El SO perdria el control del sistema.
- **El SO configura periòdicament la interrupció de rellotge per a evitar perdre el control i que un usuari pugui acaparar tots els recursos**
  - **Cada 10 ms per exemple**
  - **Típicament s'executa la planificació del sistema**

Accès al codi del kernel

- El kernel es un codi guiat per esdeveniments
  - Interrupció del flux actual d'usuari per a realitzar una tasca del SO
- Tres formes d'accedir al codi del SO (vist a EC):
  - **Interrupcions** generades pel hardware (teclat, rellotge, DMA, ...)
    - asíncrones (entre 2 instruccions de llenguatge màquina)
  - Els errors de software generen **excepcions** (divisió per zero, fallada de pàgina, ...)
    - síncrones
    - provocades per l'execució d'una instrucció de llenguatge màquina
    - es resolen (si es pot) dins de la instrucció
  - Peticions de serveo de programes: **crides a sistema**
    - síncrones
    - provocades per una instrucció explícitament (de llenguatge màquina)
    - per a demanar un servei al SO (crida al sistema)
  - Des del punt de vista hardware són molt semblants (gairabé iguals)

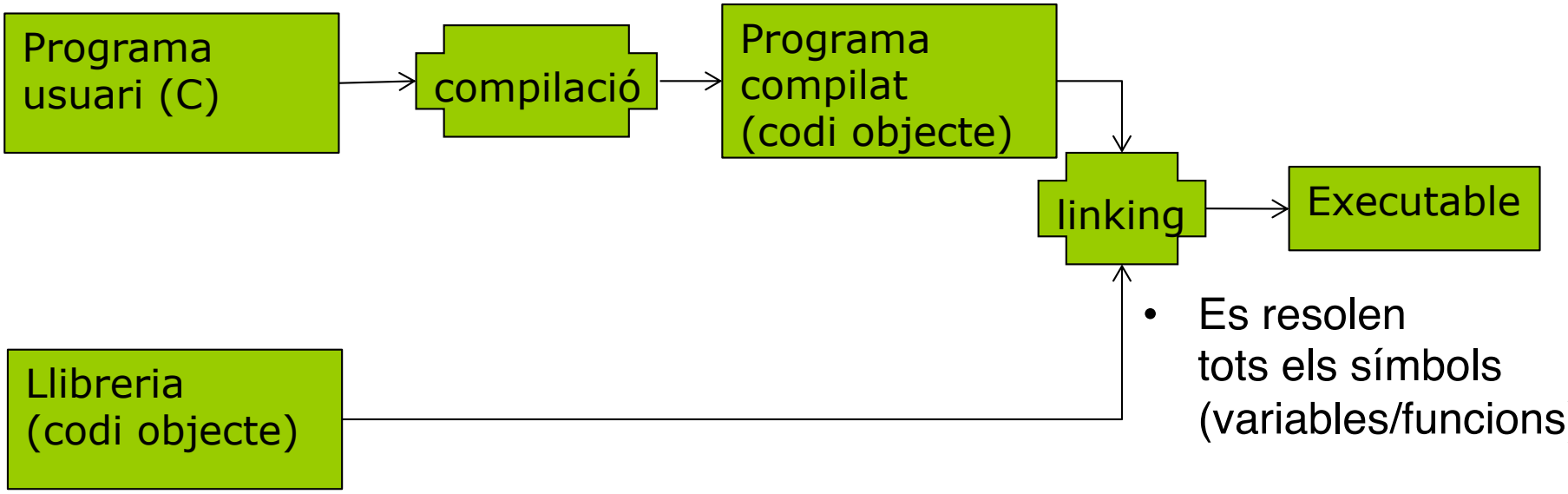
- Generació d'executables
- Requerimients de les crides a sistema
- Llibreries

CRIDES A SISTEMA



Generació d'executables

- Es comprova la sintaxis, tipus de dades, etc
- Es fa una primera generació de codi



• Llibreries: Rutines/Funcions ja compilades (és codi objecte), que "s'enllacen" amb el programa i que el programador només ha de cridar
• Poden ser a nivell de llenguatge (libc) o de sistema (libso)

Crides a Sistema

- Conjuno de **FUNCIONS** que ofereix el kernel per a accedir als seus serveis
- **Des del punt de vista del programador és igual a la interfície de qualsevol llibreria del llenguatge (C,C++, etc)**
- Normalment, els llenguatges ofereixen un API de més alt nivell que és més cómode d'utilitzar i ofereix més funcionalitats
  - Exemple: Llibreria de C: printf en comptes de write
    - Nota: La llibreria s'executa en **mode usuari** i no pot accedir al dispositiu directament
- Exemples
  - Win32 API per a Windows
  - POSIX API per a sistemes POSIX (UNIX, Linux, Max OS X)
  - Java API per a la Java Virtual Machine

Requeriments de les crides a sistema

- Requeriments
  - Des del punt de vista del programador
    - Ha de ser tan senzill com una crida a funció
      - <tipo> nombre_función(<tipo1> arg1, <tipo2> argc2..);
    - No es pot modificar el seu context (igual que en una crida a funció)
      - S'han de salvar/restaurar els registres modificats
  - Des del punt de vista del kernel necessita:
    - Execució en mode privilegiat → suport HW
    - Pas de paràmetres i retorn de resultats entre modes d'execució diferents→ depén HW
    - Les direccions que ocupen las crides a sistema han de poder ser variables per a suportar diferents versions de kernel i diferents S.O.→ per portabilitat

Solució: Llibreria “de sistema” amb suport HW

- La llibreria de sistema s’encarrega de traduïr de la funció que veu l’usuari a la petició de servei explícit al sistema
  - Passa paràmetres al kernel
  - Crida al kernel → TRAP
  - Recull resultats del kernel
  - Homogeneitza resultats (totes les crides a sistema en linux retornen -1 en cas d’error
- Com aconsseguim portabilidad entre diferents versions de SO?
  - La crida a sistema no s’identifica amb una dirección, sinó amb un identificador (un número), que utilitzem per a indexar una taula de crides a sistema (que ha de ser constant entre versions)

Crides a sistema

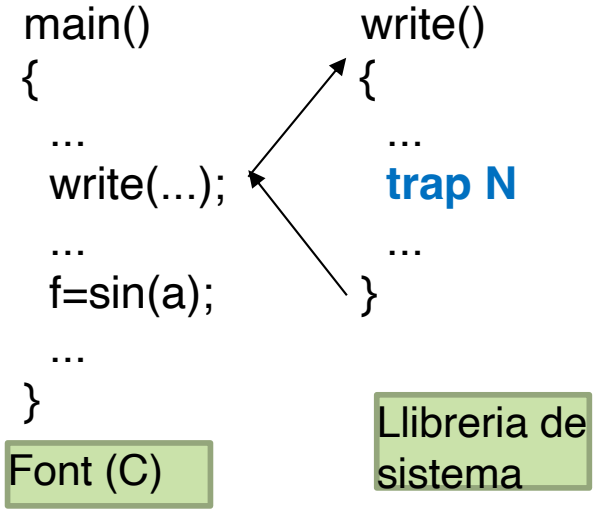
- La llibreria de sistema aïlla a l'usuari dels detalls de l'arquitectura
- El mode d'execució privilegiat ofereix seguretat: només el kernel s'executa en mode privilegiat
- La taula de crides a sistema ofereix compatibilitat entre versions de SO

- Tenim una solució que només depèn de l'arquitectura (la qual cosa és inevitable ja que és un binari amb un llenguatge màquina concret)



Llibreries “de sistema”

- El SO ofereix llibreries de sistema per a aïllar als programes d’usuari de tots els passos que s’han de fer per a
  1. Passar els paràmetres
  2. Cridar el codi del kernel
  3. Recullir i processar els resultats



- S’anomena llibreria de sistema però s’executa en **mode usuari, només serveixen per a facilitar la invocació de la crida a sistema**

Codi genèric per entrar/sortir del kernel

- Hi han passos comuns a interrupcions, excepcions i crides a sistema
- En el cas d'interrupcions i excepcions, no s'invoca explícitament ja que genera l'invocació la realitza la CPU, la resta de passos si s'aplican.

	Funció "normal"	Funció kernel
Passem els paràmetres	Push paràmetres	**DEPÈN**
Per a invocar-la	call	sysenter, int o similar
A l'inici	Salvar els registres que anem a utilitzar (push)	Salvem tots els registres (push)
Accés a paràmetres	A través de la pila: Ex: mov 8(ebp),eax	**DEPÈN**
Abans de retornar	Recuperar els registro¡es salvats en entrar (pop)	Recuperar els registres salvats (tots) en entrar (pop)
Retorn de resultats	eax (o registre equivalente)	**DEPÈN**
Per a tornar al codi que la va invocar	ret	sysexit, iret o similar